ДОМАШНЕЕ ЗАДАНИЕ 1. Дескриптивный анализ данных

Выполнил Фадеев Роман Андреевич гр. ИУ6-21М

Оглавление

Цель работы

Вариант

Задача 1. Анализ индикаторов качества государственного управления (The Worldwide Government Indicators, WGI)

1. Загрузите данные в DataFrame
2. Отсортируйте данные по убыванию score
3. Отобразите данные по индексу WGI за 2024 год в виде горизонтального столбчатого графика (score)
4. Сформируйте DataFrame из исходного для региона в соответствии с Вашим вариантом
5. Выведите данные DataFrame'a
6. Постройте графики индекса WGI за 1996-2024 для стран своего региона (estimate)
7. Найдите страны с наибольшим и наименьшим значением WGI Вашего варианта региона за 2024 год (estimate)
8. Определите средние значения региона за каждый год в период с 1996 по 2024 (estimate)
9. Постройте графики индекса WGI за 1996-2024 для стран своего региона и выделите страны с наибольшим и наименьшим значением WGI за 2024 год, а также отобразите среднее значение по региону и РФ
10. Определите, как изменилось значение показателя score с 1996 по 2024 (score)
11. Выведите таблицу для Вашего варианта (WGI - score)
12. Отобразите диаграмму размаха (boxplot) индекса WGI за 2024 для всех стран и для каждого региона в отдельности (на одном графике) (estimate)

Задача 2. Анализ рынка акций

1. Есть набор файлов, каждый из которых содержит данные по одной из акций
2. Рассчитайте корреляционную матрицу для всех акций
3. Отобразите корреляционную матрицу в виде диаграммы
4. В соответствии с Вашим вариантом определите
5. Постройте диаграммы разброса (Ваша компания - Компания с min), (Ваша компания - Компания с max), (Ваша компания - Компания с none)
6. Рассчитайте среднюю цену акций для каждого месяца (исходные данные взяты с интервалом в месяц)
7. Постройте графики для акций из пункта 4 и средней из пункта 6

Цель работы

Приобрести опыт решения практических задач по анализу данных, таких как загрузка, трансформация, вычисление простых статистик и визуализация данных в виде графиков и диаграмм, посредством языка программирования Python.

Вариант

In [1]:
```
surname = "Фадеев"  # Ваша фамилия

alp = 'абвгдеёжзийклмнопрстуфхцчшщъыьэюя'
w = [1, 42, 21, 21, 34,  6, 44, 26, 18, 44, 38, 26, 14, 43,  4, 49, 45,
```

```
        7, 42, 29,  4,  9, 36, 34, 31, 29,  5, 30,  4, 19, 28, 25, 33]

d = dict(zip(alp, w))
variant =  sum([d[el] for el in surname.lower()]) % 40 + 1

print("Задача № 1, шаг 5 - вариант: ", variant % 5 + 1)
print("задача № 2 - вариант: ", variant % 4 + 1)
```

```
Задача № 1, шаг 5 - вариант:  4
задача № 2 - вариант:  3
```

Далее определим фича флаги:

- Если `USE_PLOTLY == True`, используем библиотеку plotly для отображения графиков;
- Если `USE_PLOTLY == False`, используем matplotlib (выбрано по умолчанию).
- Если `EXPORT_MODE == True`, графики рисуются без виджетов, используется для корректного отображения в отчете;
- Если `EXPORT_MODE == False`, графики рисуются с виджетами, можно смотреть результаты по разным параметрам.

Графики plotly были добавлены, т.к. библиотека предоставляет более удобный интерфейс для построения интерактивных графиков:

- Дизайн красивый из коробки;
- Интерактивность, есть hover, zoom / pan.

В задании нет уточнения по какому измерению (Governance dimension) проводить анализ, поэтому для более удобного процесса были добавлены виджеты, которые позволяют удобно отображать таблицы/графики в зависимости от различных параметров.

In [2]:
```
USE_PLOTLY = False
EXPORT_MODE = True

if USE_PLOTLY:
    %pip install plotly
    import plotly.express as px
    import plotly.graph_objects as go
else:
    import matplotlib.pyplot as plt
```

Задача 1. Анализ индикаторов качества государственного управления (The Worldwide Government Indicators, WGI) (6 баллов)

В качестве индикатора далее необходимо использовать контроль над коррупцией (Control of Corruption) и его показатели `score` и `estimate`

- Набор данных:
  - WGI
  - Регионы
- Описание WGI

⚠️ **Замечание.** Исходный файл с данными редактировать нельзя.

1. Загрузите данные в DataFrame

In [3]:
```
%pip install openpyxl # Без предустановленного движка для открытия xsl файлов прочитать файл не удалось
import io
import requests
import pandas as pd
```

```
url = "https://www.worldbank.org/content/dam/sites/govindicators/doc/wgidataset_with_sourcedata-2025.xlsx"

headers = {
    "User-Agent": "Mozilla/5.0",
    "Accept": "*/*",
    "Referer": "https://www.worldbank.org/"
}

response = requests.get(url, headers=headers, timeout=60)
response.raise_for_status()

excel_file = io.BytesIO(response.content)

# Читаю все страницы xsl файла и делаю слияние в единый датафрейм
# При работе с датафреймом будут использоваться виджеты
# для анализа данных по конкретным измерениям (Governance dimension)
dfs = pd.read_excel(excel_file, engine="openpyxl", sheet_name=None)
# Делаем мердж всех страниц xsl & удаляем NaN записи
df = pd.concat(dfs.values(), ignore_index=True).dropna()

df.head()
```

```
Requirement already satisfied: openpyxl in /usr/local/lib/python3.11/site-packages (3.1.5)
Requirement already satisfied: et-xmlfile in /usr/local/lib/python3.11/site-packages (from openpyxl) (2.0.
0)
WARNING: Running pip as the 'root' user can result in broken permissions and conflicting behaviour with the
system package manager, possibly rendering your system unusable. It is recommended to use a virtual environ
ment instead: https://pip.pypa.io/warnings/venv. Use the --root-user-action option if you know what you are
doing and want to suppress this warning.
Note: you may need to restart the kernel to use updated packages.
```

Out[3]:

	ID variable (economy code/ gov. dimension/ year)	Economy (name)	Economy (code)	Region	Income classification	Year	Governance dimension	Number of sources	Governance estimate (approx. -2.5 to +2.5)	Standard error (estimate)
1	AFGva1996	Afghanistan	AFG	South Asia	Low income	1996	va	4	-2.235444	0.245080
2	AGOva1996	Angola	AGO	Sub-Saharan Africa	Lower middle income	1996	va	6	-1.746207	0.193985
3	ALBva1996	Albania	ALB	Europe & Central Asia	Upper middle income	1996	va	5	-0.826077	0.221397
4	AREva1996	United Arab Emirates	ARE	Middle East & North Africa	High income	1996	va	6	-0.848031	0.193985
5	ARGva1996	Argentina	ARG	Latin America & Caribbean	Upper middle income	1996	va	8	0.432919	0.193008

5 rows × 51 columns

Отсавим только необходимые колонки

In [4]:

```
columns_needed = [
    "Economy (name)",
    "Economy (code)",
    "Region",
    "Year",
    "Governance dimension",
    "Governance score (0-100)",
    "Governance estimate (approx. -2.5 to +2.5)",
```

```
    "Standard error (gov. score)"
]
```

Итоговый датафрейм

In [5]:
```
df_selected = df[columns_needed]

display(df_selected.head(20))
```

	Economy (name)	Economy (code)	Region	Year	Governance dimension	Governance score (0-100)	Governance estimate (approx. -2.5 to +2.5)	Standard error (gov. score)
1	Afghanistan	AFG	South Asia	1996	va	17.723405	-2.235444	4.289510
2	Angola	AGO	Sub-Saharan Africa	1996	va	26.286262	-1.746207	3.395228
3	Albania	ALB	Europe & Central Asia	1996	va	42.390843	-0.826077	3.874993
4	United Arab Emirates	ARE	Middle East & North Africa	1996	va	42.006593	-0.848031	3.395228
5	Argentina	ARG	Latin America & Caribbean	1996	va	64.426408	0.432919	3.378128
6	Armenia	ARM	Europe & Central Asia	1996	va	42.310930	-0.830642	4.289510
7	Antigua and Barbuda	ATG	Latin America & Caribbean	1996	va	60.143210	0.188200	5.268613
8	Australia	AUS	East Asia & Pacific	1996	va	81.698045	1.419730	3.380890
9	Austria	AUT	Europe & Central Asia	1996	va	81.965561	1.435014	3.380890
10	Azerbaijan	AZE	Europe & Central Asia	1996	va	33.280657	-1.346585	3.663736
11	Burundi	BDI	Sub-Saharan Africa	1996	va	36.057534	-1.187928	4.289510
12	Belgium	BEL	Europe & Central Asia	1996	va	82.384636	1.458958	3.380890
13	Benin	BEN	Sub-Saharan Africa	1996	va	59.513789	0.152238	4.289510
14	Burkina Faso	BFA	Sub-Saharan Africa	1996	va	45.438931	-0.651925	3.874993
15	Bangladesh	BGD	South Asia	1996	va	51.285371	-0.317890	3.395228
16	Bulgaria	BGR	Europe & Central Asia	1996	va	61.319230	0.255392	3.395228
17	Bahrain	BHR	Middle East & North Africa	1996	va	37.750429	-1.091205	3.395228
18	Bahamas, The	BHS	Latin America & Caribbean	1996	va	76.082483	1.098886	4.551353
19	Bosnia and Herzegovina	BIH	Europe & Central Asia	1996	va	53.455626	-0.193893	3.796046
20	Belarus	BLR	Europe & Central Asia	1996	va	40.938047	-0.909082	4.289510

2. Отсортируйте данные по убыванию `score`

In [6]:
```
from IPython.display import display
import ipywidgets as widgets

target_year = 2024


dimensions = df_selected["Governance dimension"].unique()

dimension_dropdown = widgets.Dropdown(
    options=dimensions,
    description='Dimension:',
    value=dimensions[0],
    style={'description_width': 'initial'},
    layout=widgets.Layout(width='400px')
)

year_dropdown = widgets.Dropdown(
    options=sorted(df_selected["Year"].unique(), reverse=True),
    description='Year:',
    value=target_year,
    style={'description_width': 'initial'},
    layout=widgets.Layout(width='200px')
)

from IPython.display import display
import ipywidgets as widgets

target_year = 2024


dimensions = df_selected["Governance dimension"].unique()

dimension_dropdown = widgets.Dropdown(
    options=dimensions,
    description='Dimension:',
    value=dimensions[0],
    style={'description_width': 'initial'},
    layout=widgets.Layout(width='400px')
)

year_dropdown = widgets.Dropdown(
    options=sorted(df_selected["Year"].unique(), reverse=True),
    description='Year:',
    value=target_year,
    style={'description_width': 'initial'},
    layout=widgets.Layout(width='200px')
)

def get_table(dimension, year):
    return df_selected[
        (df_selected["Year"] == year) &
        (df_selected["Governance dimension"] == dimension)
    ].sort_values(by="Governance score (0-100)", ascending=False).head(20)


if EXPORT_MODE:
    df_static = get_table(dimension_dropdown.value, year_dropdown.value)
    display(df_static)
else:
    def update_table(dimension, year):
        display(get_table(dimension, year))

    out = widgets.interactive_output(update_table, {
        'dimension': dimension_dropdown,
        'year': year_dropdown
    })

    display(widgets.HBox([dimension_dropdown, year_dropdown]), out)
```

	Economy (name)	Economy (code)	Region	Year	Governance dimension	Governance score (0-100)	Governance estimate (approx. -2.5 to +2.5)	Standard error (gov. score)
5375	Norway	NOR	Europe & Central Asia	2024	va	90.618972	1.929425	2.952238
5285	Denmark	DNK	Europe & Central Asia	2024	va	90.015415	1.894940	2.886693
5294	Finland	FIN	Europe & Central Asia	2024	va	89.291120	1.853558	2.886693
5412	Sweden	SWE	Europe & Central Asia	2024	va	87.089942	1.727794	2.854150
5378	New Zealand	NZL	East Asia & Pacific	2024	va	86.415042	1.689234	2.878536
5347	Luxembourg	LUX	Europe & Central Asia	2024	va	85.996619	1.665328	2.886693
5374	Netherlands	NLD	Europe & Central Asia	2024	va	85.792463	1.653663	2.886693
5266	Switzerland	CHE	Europe & Central Asia	2024	va	85.592487	1.642238	3.597008
5320	Ireland	IRL	Europe & Central Asia	2024	va	85.512462	1.637665	2.886693
5282	Germany	DEU	Europe & Central Asia	2024	va	84.820868	1.598151	2.854150
5244	Australia	AUS	East Asia & Pacific	2024	va	83.318782	1.512330	2.873182
5265	Canada	CAN	North America	2024	va	83.166055	1.503604	2.977704
5248	Belgium	BEL	Europe & Central Asia	2024	va	83.054204	1.497214	2.886693
5323	Iceland	ISL	Europe & Central Asia	2024	va	81.786448	1.424781	3.597008
5292	Estonia	EST	Europe & Central Asia	2024	va	81.606500	1.414499	2.679743
5245	Austria	AUT	Europe & Central Asia	2024	va	81.448237	1.405457	2.886693
5308	Greenland	GRL	Europe & Central Asia	2024	va	81.434893	1.404695	6.551841
5343	Liechtenstein	LIE	Europe & Central Asia	2024	va	80.086281	1.327642	4.794924
5299	United Kingdom	GBR	Europe & Central Asia	2024	va	79.613180	1.300612	2.854998

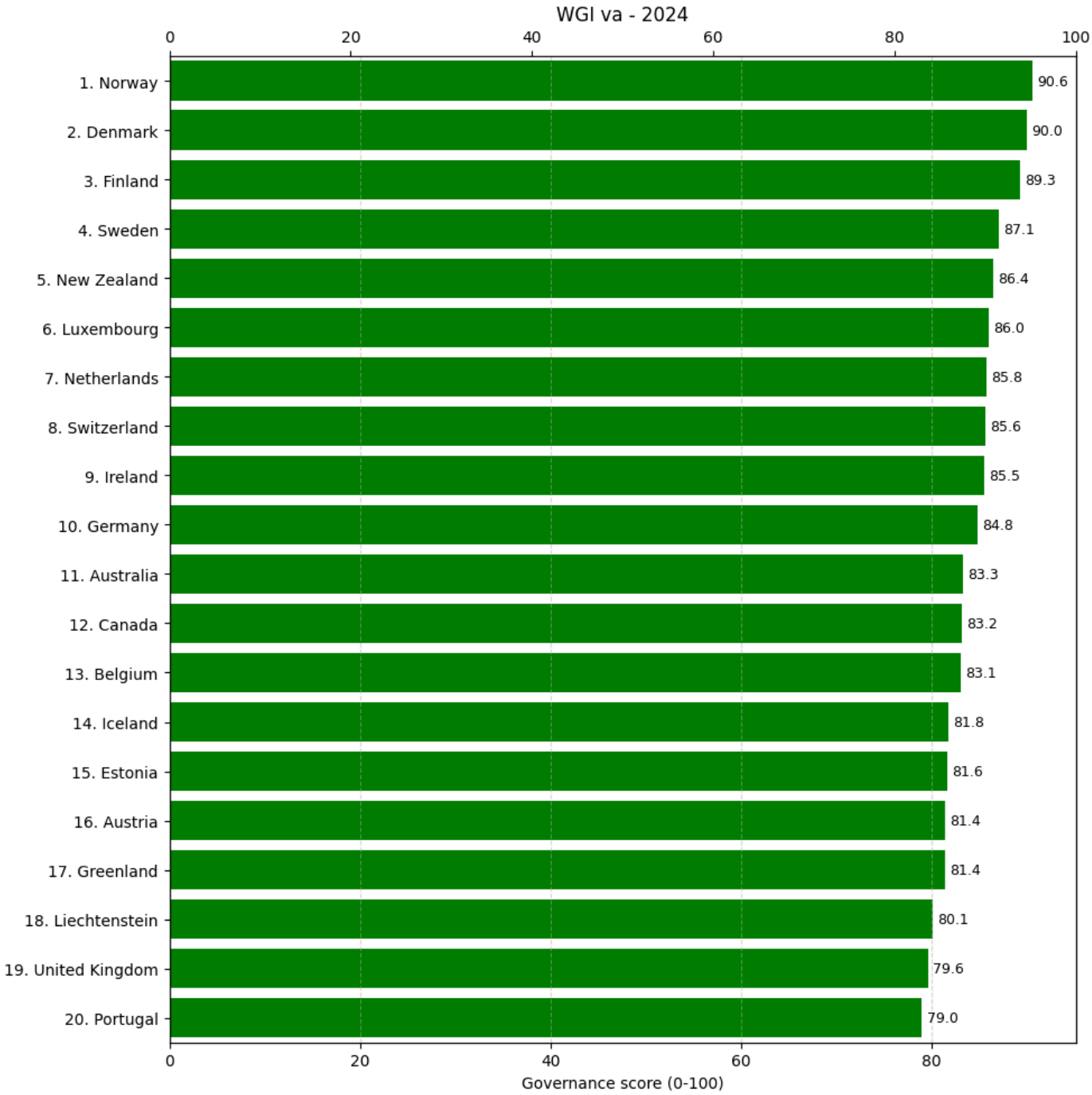
	Economy (name)	Economy (code)	Region	Year	Governance dimension	Governance score (0-100)	Governance estimate (approx. -2.5 to +2.5)	Standard error (gov. score)
5389	Portugal	PRT	Europe & Central Asia	2024	va	78.992314	1.265139	2.854150

3. Отобразите данные по индексу WGI за 2024 год в виде горизонтального столбчатого графика (`score`). Примерный вид графика приведен ниже

No description has been provided for this image

In [7]:
```
import matplotlib.pyplot as plt

dimension_dropdown = widgets.Dropdown(
    options=dimensions,
    description='Dimension:',
    value=dimensions[0],
    style={'description_width': 'initial'},
    layout=widgets.Layout(width='400px')
)

year_dropdown = widgets.Dropdown(
    options=sorted(df_selected["Year"].unique(), reverse=True),
    description='Year:',
    value=2024,
    style={'description_width': 'initial'},
    layout=widgets.Layout(width='200px')
)

top_n_slider = widgets.IntSlider(
    value=20,
    min=5,
    max=100,
    step=5,
    description='Top N:',
    style={'description_width': 'initial'},
    layout=widgets.Layout(width='300px')
)

def plot_wgi(dimension, year, top_n):
    df_filtered = df_selected[
        (df_selected["Year"] == year) &
        (df_selected["Governance dimension"] == dimension)
    ].sort_values(by="Governance score (0-100)", ascending=False).head(top_n)

    df_filtered = df_filtered[::-1]

    plt.figure(figsize=(10, max(5, len(df_filtered)/2)))

    bars = plt.barh(range(len(df_filtered)), df_filtered["Governance score (0-100)"], color='green')

    plt.yticks(
        range(len(df_filtered)),
        [f"{len(df_filtered)-i}. {c}" for i, c in enumerate(df_filtered["Economy (name)"])]
    )

    ax = plt.gca()
    ax.set_ylim(-0.5, len(df_filtered)-0.5)

    ax_top = ax.twiny()
    ax_top.set_xlim(ax.get_xlim())
    ax_top.set_xticks(ax.get_xticks())
    ax_top.set_xticklabels([str(int(x)) for x in ax.get_xticks()])

    for i, v in enumerate(df_filtered["Governance score (0-100)"]):
        ax.text(v + 0.5, i, f"{v:.1f}", va='center', ha='left', fontsize=9, color='black')
```

```python
    ax.set_xlabel("Governance score (0-100)")
    plt.title(f"WGI {dimension} - {year}")
    ax.grid(axis='x', linestyle='--', alpha=0.5)

    plt.tight_layout()
    plt.margins(y=0)

    plt.show()

if EXPORT_MODE:
    plot_wgi(
        dimension_dropdown.value,
        year_dropdown.value,
        top_n_slider.value
    )

else:
    out = widgets.interactive_output(plot_wgi, {
        'dimension': dimension_dropdown,
        'year': year_dropdown,
        'top_n': top_n_slider
    })

    display(widgets.HBox([dimension_dropdown, year_dropdown, top_n_slider]), out)
```



4. Сформируйте DataFrame из исходного для региона в соответствии с Вашим вариантом

- Вариант 1. Asia Pacific
- Вариант 2. Americas
- Вариант 3. Europe and Central Asia
- Вариант 4. Middle East and North Africa
- Вариант 5. Sub Saharan Africa

Вариант 4. Middle East and North Africa

In [8]:
```
df_regions = pd.read_excel("./data/regions.xlsx")
```

In [9]:
```
region_code = "MENA"
countries_codes_in_region = df_regions[df_regions["Region"] == region_code]["Code"].tolist()
```

5. Выведите данные DataFrame'a

In [10]:
```
df_filtered_region = df_selected[df_selected["Economy (code)"].isin(countries_codes_in_region)]
print(df_filtered_region["Economy (name)"].unique())
df_filtered_region.head()
```

```
<ArrowStringArray>
['United Arab Emirates',              'Bahrain',              'Algeria',
     'Egypt, Arab Rep.',   'Iran, Islamic Rep.',                 'Iraq',
               'Israel',               'Jordan',               'Kuwait',
              'Lebanon',                'Libya',              'Morocco',
                 'Oman',                'Qatar',         'Saudi Arabia',
 'Syrian Arab Republic',              'Tunisia',          'Yemen, Rep.']
Length: 18, dtype: str
```

Out[10]:

	Economy (name)	Economy (code)	Region	Year	Governance dimension	Governance score (0-100)	Governance estimate (approx. -2.5 to +2.5)	Standard error (gov. score)
4	United Arab Emirates	ARE	Middle East & North Africa	1996	va	42.006593	-0.848031	3.395228
17	Bahrain	BHR	Middle East & North Africa	1996	va	37.750429	-1.091205	3.395228
51	Algeria	DZA	Middle East & North Africa	1996	va	36.730597	-1.149473	3.395228
53	Egypt, Arab Rep.	EGY	Middle East & North Africa	1996	va	36.771847	-1.147116	3.395228
83	Iran, Islamic Rep.	IRN	Middle East & North Africa	1996	va	36.559318	-1.159259	3.395228

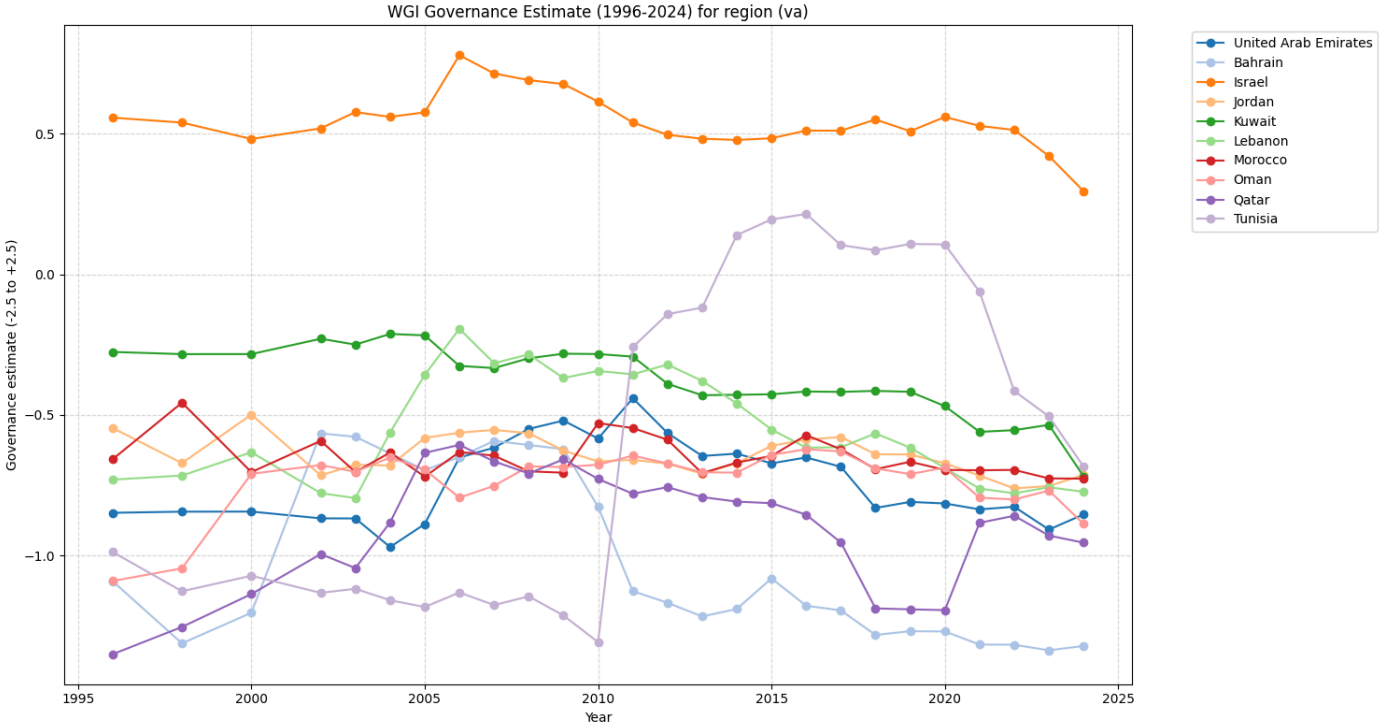
6. Постройте графики индекса WGI за 1996-2024 для стран своего региона (`estimate`). Примерный вид графика приведен ниже

No description has been provided for this image

In [11]:
```
dimensions = df_filtered_region["Governance dimension"].unique()

dimension_dropdown = widgets.Dropdown(
    options=dimensions,
    description='Dimension:',
    value=dimensions[0],
    style={'description_width': 'initial'},
```

```python
    layout=widgets.Layout(width='400px')
)

top_n_slider = widgets.IntSlider(
    value=10,
    min=1,
    max=min(50, df_filtered_region["Economy (name)"].nunique()),
    step=1,
    description='Top N:',
    style={'description_width': 'initial'},
    layout=widgets.Layout(width='300px')
)

def plot_region_dimension(dimension, top_n):
    df_plot = df_filtered_region[
        df_filtered_region["Governance dimension"] == dimension
    ][["Economy (name)", "Year", "Governance estimate (approx. -2.5 to +2.5)"]]

    top_countries = (
        df_plot.groupby("Economy (name)")["Governance estimate (approx. -2.5 to +2.5)"]
        .max()
        .sort_values(ascending=False)
        .head(top_n)
        .index
    )
    df_plot = df_plot[df_plot["Economy (name)"].isin(top_countries)]

    if USE_PLOTLY:
        fig = px.line(
            df_plot,
            x="Year",
            y="Governance estimate (approx. -2.5 to +2.5)",
            color="Economy (name)",
            markers=True,
            hover_data={
                "Economy (name)": True,
                "Year": True,
                "Governance estimate (approx. -2.5 to +2.5)": ':.2f'
            }
        )

        fig.update_layout(
            title=f"WGI Governance Estimate (1996-2024) for region ({dimension})",
            xaxis_title="Year",
            yaxis_title="Governance estimate (-2.5 to +2.5)",
            legend_title="Country",
            template="plotly_white"
        )

        fig.show()

    else:
        plt.figure(figsize=(15, 8))

        countries = df_plot["Economy (name)"].unique()
        base_colors = plt.cm.tab20.colors
        colors = [base_colors[i % 20] for i in range(len(countries))]

        for i, country in enumerate(countries):
            df_country = df_plot[df_plot["Economy (name)"] == country].sort_values("Year")

            plt.plot(
                df_country["Year"],
                df_country["Governance estimate (approx. -2.5 to +2.5)"],
                marker='o',
                linestyle='-',
                label=country,
                color=colors[i]
            )

        plt.xlabel("Year")
        plt.ylabel("Governance estimate (-2.5 to +2.5)")
        plt.title(f"WGI Governance Estimate (1996-2024) for region ({dimension})")
```

```
        plt.grid(True, linestyle='--', alpha=0.5)
        plt.legend(bbox_to_anchor=(1.05, 1), loc='upper left')

        plt.tight_layout()
        plt.show()


if EXPORT_MODE:
    plot_region_dimension(
        dimension_dropdown.value,
        top_n_slider.value
    )

else:
    out = widgets.interactive_output(plot_region_dimension, {
        'dimension': dimension_dropdown,
        'top_n': top_n_slider
    })

    display(widgets.HBox([dimension_dropdown, top_n_slider]), out)
```



7. Найдите страны с наибольшим и наименьшим значением WGI Вашего варианта региона за 2024 год (`estimate`)

> ⚠️ **Замечание.** У нескольких стран может быть одна и та же позиция в рейтинге из-за одинаковых значений индекса

8. Определите средние значения региона за каждый год в период с 1996 по 2024 (`estimate`)

In [12]:
```
dimensions = df_filtered_region["Governance dimension"].unique()
years = sorted(df_filtered_region["Year"].unique(), reverse=True)

dimension_dropdown = widgets.Dropdown(
    options=dimensions,
    description='Dimension:',
    value=dimensions[0],
    style={'description_width': 'initial'},
    layout=widgets.Layout(width='400px')
)

year_dropdown = widgets.Dropdown(
    options=years,
```

```python
    description='Year:',
    value=2024,
    style={'description_width': 'initial'},
    layout=widgets.Layout(width='200px')
)

def compute_wgi_stats(dimension, year):
    df_dim = df_filtered_region[
        df_filtered_region["Governance dimension"] == dimension
    ]

    df_year = df_dim[
        df_dim["Year"] == year
    ][["Economy (name)", "Governance estimate (approx. -2.5 to +2.5)"]]

    max_value = df_year["Governance estimate (approx. -2.5 to +2.5)"].max()
    min_value = df_year["Governance estimate (approx. -2.5 to +2.5)"].min()

    max_countries = df_year[
        df_year["Governance estimate (approx. -2.5 to +2.5)"] == max_value
    ]["Economy (name)"].tolist()

    min_countries = df_year[
        df_year["Governance estimate (approx. -2.5 to +2.5)"] == min_value
    ]["Economy (name)"].tolist()

    df_region_estimate = df_dim[["Year", "Governance estimate (approx. -2.5 to +2.5)"]]
    region_mean_per_year = (
        df_region_estimate
        .groupby("Year")
        .mean()
        .reset_index()
        .rename(columns={
            "Governance estimate (approx. -2.5 to +2.5)": "Regional Mean Estimate"
        })
    )

    return max_value, min_value, max_countries, min_countries, region_mean_per_year


def show_wgi_stats(dimension, year):
    max_val, min_val, max_c, min_c, df_mean = compute_wgi_stats(dimension, year)

    print(f"Страны с наибольшим WGI за {year} (Estimate = {max_val:.2f}): {', '.join(max_c)}")
    print(f"Страны с наименьшим WGI за {year} (Estimate = {min_val:.2f}): {', '.join(min_c)}\n")

    display(df_mean)


if EXPORT_MODE:
    show_wgi_stats(
        dimension_dropdown.value,
        year_dropdown.value
    )

else:
    out = widgets.interactive_output(show_wgi_stats, {
        'dimension': dimension_dropdown,
        'year': year_dropdown
    })

    display(widgets.HBox([dimension_dropdown, year_dropdown]), out)
```

```
Страны с наибольшим WGI за 2024 (Estimate = 0.30): Israel
Страны с наименьшим WGI за 2024 (Estimate = -1.85): Syrian Arab Republic
```

	Year	Regional Mean Estimate
0	1996	-1.045856
1	1998	-1.080369
2	2000	-1.049152
3	2002	-0.995664
4	2003	-0.983702
5	2004	-0.917957
6	2005	-0.870692
7	2006	-0.824171
8	2007	-0.834705
9	2008	-0.841918
10	2009	-0.858823
11	2010	-0.896225
12	2011	-0.846712
13	2012	-0.809041
14	2013	-0.852651
15	2014	-0.848534
16	2015	-0.846341
17	2016	-0.875042
18	2017	-0.895195
19	2018	-0.941168
20	2019	-0.941538
21	2020	-0.954668
22	2021	-0.984874
23	2022	-1.010110
24	2023	-1.029165
25	2024	-1.054704

9. Постройте графики индекса WGI за 1996-2024 для стран своего региона и выделите страны с наибольшим и наименьшим значением WGI за 2024 год, а также отобразите среднее значение по региону и РФ. Примерный вид графика приведен ниже

No description has been provided for this image

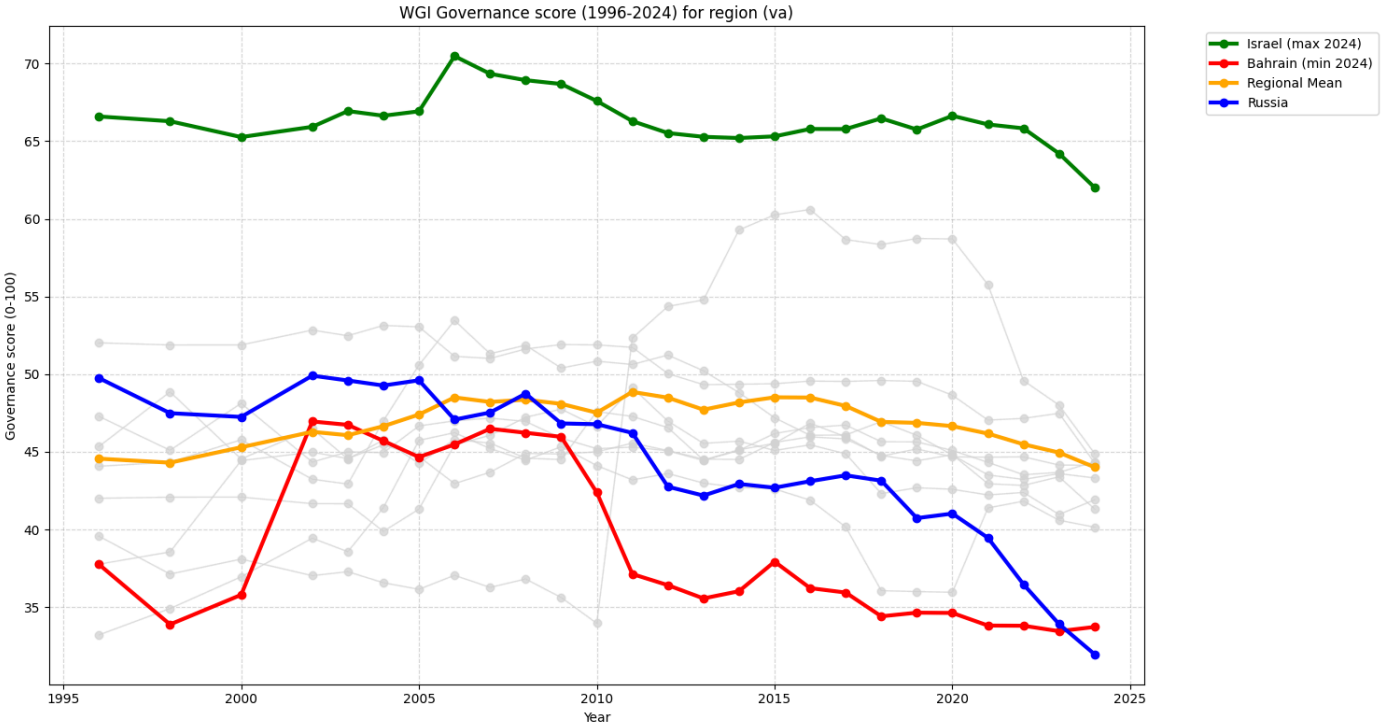
График построен по параметру `Governance score (0-100)`, для параметра `Governance estimate (approx. -2.5 to +2.5)` результат +- аналогичен.

In [13]:
```
dimensions = df_filtered_region["Governance dimension"].unique()

dimension_dropdown = widgets.Dropdown(
    options=dimensions,
    description='Dimension:',
    value=dimensions[0],
    style={'description_width': 'initial'},
    layout=widgets.Layout(width='400px')
)

top_n_slider = widgets.IntSlider(
```

```python
    value=10,
    min=1,
    max=min(50, df_filtered_region["Economy (name)"].nunique()),
    step=1,
    description='Top N:',
    style={'description_width': 'initial'},
    layout=widgets.Layout(width='300px')
)

def plot_region_dimension(dimension, top_n):
    df_plot = df_filtered_region[
        df_filtered_region["Governance dimension"] == dimension
    ][["Economy (name)", "Year", "Governance score (0-100)"]]

    top_countries = (
        df_plot.groupby("Economy (name)")["Governance score (0-100)"]
        .max().sort_values(ascending=False).head(top_n).index
    )
    df_plot = df_plot[df_plot["Economy (name)"].isin(top_countries)]

    df_2024 = df_plot[df_plot["Year"] == 2024]
    max_val = df_2024["Governance score (0-100)"].max()
    min_val = df_2024["Governance score (0-100)"].min()

    max_countries = df_2024[df_2024["Governance score (0-100)"] == max_val]["Economy (name)"].tolist()
    min_countries = df_2024[df_2024["Governance score (0-100)"] == min_val]["Economy (name)"].tolist()

    if USE_PLOTLY:
        fig = go.Figure()

        gray_countries = [
            c for c in df_plot["Economy (name)"].unique() if c not in max_countries + min_countries
        ]

        for country in gray_countries:
            df_country = df_plot[df_plot["Economy (name)"] == country].sort_values("Year")
            fig.add_trace(go.Scatter(
                x=df_country["Year"],
                y=df_country["Governance score (0-100)"],
                mode='lines+markers',
                line=dict(color='lightgray'),
                marker=dict(color='lightgray'),
                name=country,
                showlegend=False
            ))

        for country in max_countries:
            df_country = df_plot[df_plot["Economy (name)"] == country].sort_values("Year")
            fig.add_trace(go.Scatter(
                x=df_country["Year"],
                y=df_country["Governance score (0-100)"],
                mode='lines+markers',
                line=dict(width=3, color='green'),
                name=f"{country} (max 2024)"
            ))

        for country in min_countries:
            df_country = df_plot[df_plot["Economy (name)"] == country].sort_values("Year")
            fig.add_trace(go.Scatter(
                x=df_country["Year"],
                y=df_country["Governance score (0-100)"],
                mode='lines+markers',
                line=dict(width=3, color='red'),
                name=f"{country} (min 2024)"
            ))

        df_mean = df_plot.groupby("Year")["Governance score (0-100)"].mean().reset_index()
        fig.add_trace(go.Scatter(
            x=df_mean["Year"],
            y=df_mean["Governance score (0-100)"],
            mode='lines+markers',
            line=dict(width=3, color='yellow'),
            name="Regional Mean"
```

```
        ))

        df_russia = df_selected[
            (df_selected["Economy (name)"] == "Russian Federation") &
            (df_selected["Governance dimension"] == dimension)
        ][["Year", "Governance score (0-100)"]].sort_values("Year")

        if not df_russia.empty:
            fig.add_trace(go.Scatter(
                x=df_russia["Year"],
                y=df_russia["Governance score (0-100)"],
                mode='lines+markers',
                line=dict(width=3, color='blue'),
                name="Russia"
            ))

        fig.update_layout(
            title=f"WGI Governance score (1996-2024) for region ({dimension})",
            xaxis_title="Year",
            yaxis_title="Governance score (0-100)",
            template="plotly_white"
        )

        fig.show()

    else:
        plt.figure(figsize=(15, 8))

        gray_countries = [
            c for c in df_plot["Economy (name)"].unique() if c not in max_countries + min_countries
        ]

        for country in gray_countries:
            df_country = df_plot[df_plot["Economy (name)"] == country].sort_values("Year")
            plt.plot(
                df_country["Year"],
                df_country["Governance score (0-100)"],
                color='lightgray',
                marker='o',
                linewidth=1,
                alpha=0.7
            )

        for country in max_countries:
            df_country = df_plot[df_plot["Economy (name)"] == country].sort_values("Year")
            plt.plot(
                df_country["Year"],
                df_country["Governance score (0-100)"],
                color='green',
                marker='o',
                linewidth=3,
                label=f"{country} (max 2024)"
            )

        for country in min_countries:
            df_country = df_plot[df_plot["Economy (name)"] == country].sort_values("Year")
            plt.plot(
                df_country["Year"],
                df_country["Governance score (0-100)"],
                color='red',
                marker='o',
                linewidth=3,
                label=f"{country} (min 2024)"
            )

        df_mean = df_plot.groupby("Year")["Governance score (0-100)"].mean().reset_index()
        plt.plot(
            df_mean["Year"],
            df_mean["Governance score (0-100)"],
            color='orange',
            linewidth=3,
            marker='o',
            label="Regional Mean"
```

```
        )

        df_russia = df_selected[
            (df_selected["Economy (name)"] == "Russian Federation") &
            (df_selected["Governance dimension"] == dimension)
        ][["Year", "Governance score (0-100)"]].sort_values("Year")

        if not df_russia.empty:
            plt.plot(
                df_russia["Year"],
                df_russia["Governance score (0-100)"],
                color='blue',
                linewidth=3,
                marker='o',
                label="Russia"
            )

        plt.title(f"WGI Governance score (1996-2024) for region ({dimension})")
        plt.xlabel("Year")
        plt.ylabel("Governance score (0-100)")
        plt.grid(True, linestyle='--', alpha=0.5)

        plt.legend(bbox_to_anchor=(1.05, 1), loc='upper left')
        plt.tight_layout()
        plt.show()

if EXPORT_MODE:
    plot_region_dimension(dimension_dropdown.value, top_n_slider.value)
else:
    out = widgets.interactive_output(plot_region_dimension, {
        'dimension': dimension_dropdown,
        'top_n': top_n_slider
    })
    display(widgets.HBox([dimension_dropdown, top_n_slider]), out)
```



10. Определите, как изменилось значение показателя `score` с 1996 по 2024 (`score`)

Изменение показателя `score` представлено выше на графике. Также сравнение WGI 1996 с WGI 2024 приведено ниже в таблице.

11. Выведите таблицу для ***Вашего варианта*** (WGI - `score`)

	Регион	Страна	WGI 1996	WGI 2024	Изменение (см. пункт 10)
mean_2024	-	-	-	-	-
max_2024	-	-	-	-	-
min_2024	-	-	-	-	-
Russia_2024	-	-	-	-	-

In [14]:

```
dimensions = df_selected["Governance dimension"].unique()

dimension_dropdown = widgets.Dropdown(
    options=dimensions,
    description='Dimension:',
    value=dimensions[0],
    style={'description_width': 'initial'},
    layout=widgets.Layout(width='400px')
)

def update_summary_region(dimension):
    df_region_score = df_filtered_region[df_filtered_region["Governance dimension"] == dimension][
        ["Economy (name)", "Region", "Year", "Governance score (0-100)"]
    ]

    df_russia = df_selected[
        (df_selected["Economy (name)"] == "Russian Federation") &
        (df_selected["Governance dimension"] == dimension)
    ][["Economy (name)", "Region", "Year", "Governance score (0-100)"]]

    df_score = pd.concat([df_region_score, df_russia], ignore_index=True)

    df_1996 = df_score[df_score["Year"] == 1996][[
        "Economy (name)",
        "Region",
        "Governance score (0-100)"]
    ].rename(
        columns={"Governance score (0-100)": "WGI 1996"}
    )
    df_2024 = df_score[df_score["Year"] == 2024][[
        "Economy (name)",
        "Region",
        "Governance score (0-100)"]
    ].rename(
        columns={"Governance score (0-100)": "WGI 2024"}
    )

    df_combined = pd.merge(df_1996, df_2024, on=["Economy (name)", "Region"], how="outer")
    df_combined["Изменение (см. пункт 10)"] = df_combined["WGI 2024"] - df_combined["WGI 1996"]

    rows = []

    mean_1996 = df_region_score[df_region_score["Year"] == 1996]["Governance score (0-100)"].mean()
    mean_2024 = df_region_score[df_region_score["Year"] == 2024]["Governance score (0-100)"].mean()
    rows.append({
        "": "mean",
        "Регион": "-",
        "Страна": "-",
        "WGI 1996": mean_1996,
        "WGI 2024": mean_2024,
        "Изменение (см. пункт 10)": mean_2024 - mean_1996
    })

    max_val = df_region_score[df_region_score["Year"] == 2024]["Governance score (0-100)"].max()
    max_rows = df_region_score[
        (df_region_score["Year"] == 2024) &
        (df_region_score["Governance score (0-100)"] == max_val)
    ]
    for _, r in max_rows.iterrows():
        row_1996 = df_combined[
            df_combined["Economy (name)"] == r["Economy (name)"]
        ]["WGI 1996"].values[0]
        rows.append({
```

```
            "": "max",
            "Регион": r["Region"],
            "Страна": r["Economy (name)"],
            "WGI 1996": row_1996,
            "WGI 2024": r["Governance score (0-100)"],
            "Изменение (см. пункт 10)": r["Governance score (0-100)"] - row_1996
        })

    min_val = df_region_score[df_region_score["Year"] == 2024]["Governance score (0-100)"].min()
    min_rows = df_region_score[
        (df_region_score["Year"] == 2024) &
        (df_region_score["Governance score (0-100)"] == min_val)
    ]
    for _, r in min_rows.iterrows():
        row_1996 = df_combined[
            df_combined["Economy (name)"] == r["Economy (name)"]
        ]["WGI 1996"].values[0]
        rows.append({
            "": "min",
            "Регион": r["Region"],
            "Страна": r["Economy (name)"],
            "WGI 1996": row_1996,
            "WGI 2024": r["Governance score (0-100)"],
            "Изменение (см. пункт 10)": r["Governance score (0-100)"] - row_1996
        })

    if not df_russia.empty:
        r = df_russia[df_russia["Year"] == 2024].iloc[0]
        row_1996 = df_combined[df_combined["Economy (name)"] == r["Economy (name)"]]["WGI 1996"].values[0]
        rows.append({
            "": "Russia",
            "Регион": r["Region"],
            "Страна": "Russia",
            "WGI 1996": row_1996,
            "WGI 2024": r["Governance score (0-100)"],
            "Изменение (см. пункт 10)": r["Governance score (0-100)"] - row_1996
        })

    df_summary = pd.DataFrame(rows)
    display(df_summary)

if EXPORT_MODE:
    update_summary_region(dimension_dropdown.value)
else:
    out = widgets.interactive_output(update_summary_region, {'dimension': dimension_dropdown})
    display(dimension_dropdown, out)
```

		Регион	Страна	WGI 1996	WGI 2024	Изменение (см. пункт 10)
0	mean	-	-	38.544163	38.389295	-0.154868
1	max	Middle East & North Africa	Israel	66.591641	62.015206	-4.576435
2	min	Middle East & North Africa	Syrian Arab Republic	28.660460	24.517035	-4.143425
3	Russia	Europe & Central Asia	Russia	49.747270	31.957630	-17.789640

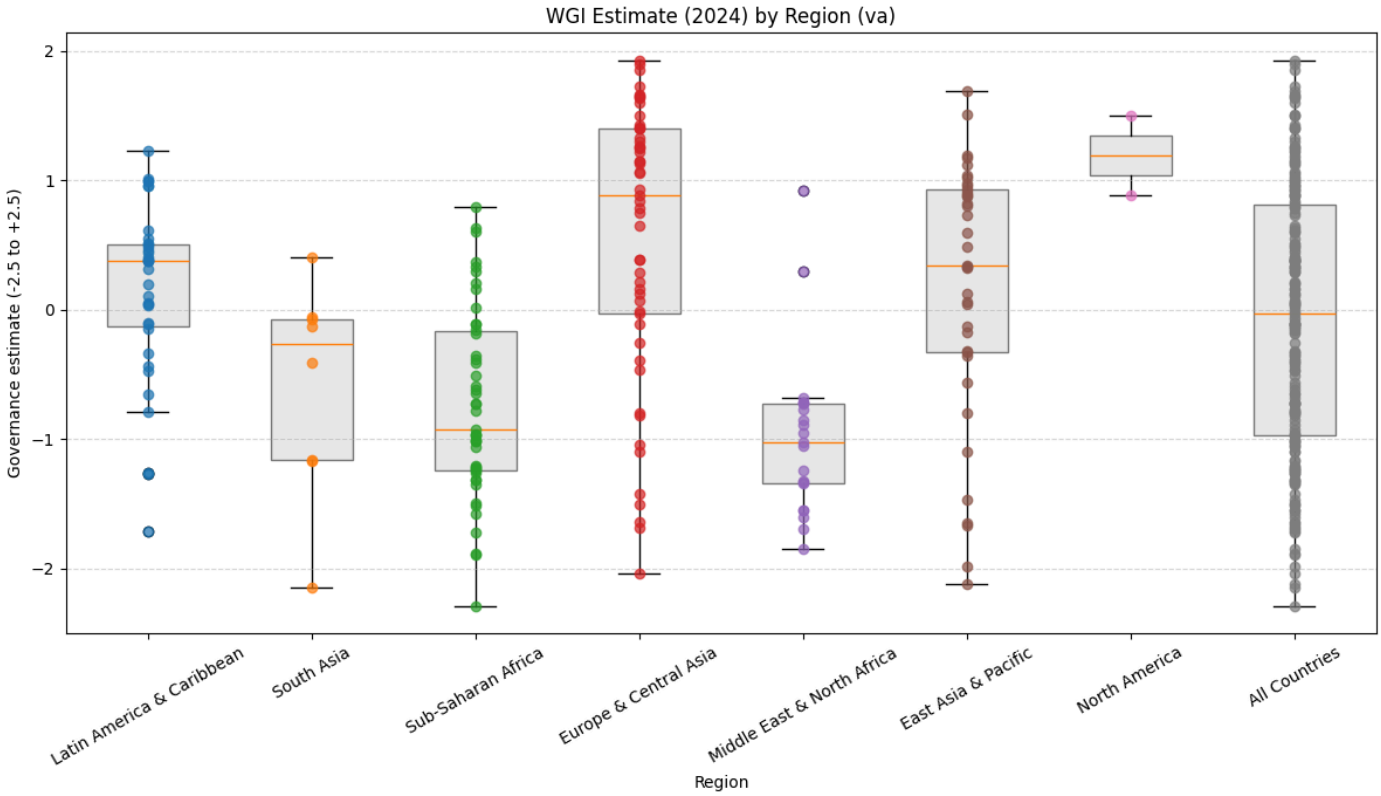
12. Отобразите диаграмму размаха (`boxplot`) индекса WGI за 2024 для всех стран и для каждого региона в отдельности (на одном графике) (`estimate`)

In [15]:
```
dimensions = df_selected["Governance dimension"].unique()

dimension_dropdown = widgets.Dropdown(
    options=dimensions,
    description='Dimension:',
    value=dimensions[0],
    style={'description_width': 'initial'},
    layout=widgets.Layout(width='400px')
)
```

```python
def plot_wgi_boxplot(dimension):
    df_plot = df_selected[
        (df_selected["Governance dimension"] == dimension) &
        (df_selected["Year"] == 2024)
    ][["Economy (name)", "Region", "Governance estimate (approx. -2.5 to +2.5)"]].copy()

    df_plot_all = df_plot.copy()
    df_plot_all["Region"] = "All Countries"

    df_box = pd.concat([df_plot, df_plot_all], ignore_index=True)

    if USE_PLOTLY:
        fig = px.box(
            df_box,
            x="Region",
            y="Governance estimate (approx. -2.5 to +2.5)",
            points="all",
            color="Region",
            hover_data={
                "Economy (name)": True,
                "Region": True,
                "Governance estimate (approx. -2.5 to +2.5)": ':.2f'
            },
            title=f"WGI Estimate (2024) by Region ({dimension})",
            labels={"Governance estimate (approx. -2.5 to +2.5)": "Governance estimate (-2.5 to +2.5)"},
            template="plotly_white"
        )

        fig.update_layout(showlegend=False)
        fig.show()

    else:
        plt.figure(figsize=(12, 7))

        regions = df_box["Region"].unique()
        data = []
        labels = []

        for region in regions:
            values = df_box[df_box["Region"] == region]["Governance estimate (approx. -2.5 to +2.5)"]
            data.append(values)
            labels.append(region)

        box = plt.boxplot(
            data,
            labels=labels,
            patch_artist=True,
            zorder=1
        )

        for patch in box['boxes']:
            patch.set_facecolor('lightgray')
            patch.set_alpha(0.5)

        for i, region in enumerate(regions):
            df_region = df_box[df_box["Region"] == region]
            y = df_region["Governance estimate (approx. -2.5 to +2.5)"]
            x = [i + 1] * len(y)

            plt.scatter(
                x, y,
                alpha=0.7,
                zorder=3
            )

        plt.title(f"WGI Estimate (2024) by Region ({dimension})")
        plt.xlabel("Region")
        plt.ylabel("Governance estimate (-2.5 to +2.5)")
        plt.grid(axis='y', linestyle='--', alpha=0.5)

        plt.xticks(rotation=30)
        plt.tight_layout()
        plt.show()
```

```
if EXPORT_MODE:
    plot_wgi_boxplot(dimension_dropdown.value)
else:
    out = widgets.interactive_output(plot_wgi_boxplot, {'dimension': dimension_dropdown})
    display(dimension_dropdown, out)
```

```
/tmp/ipykernel_5844/3855492300.py:54: MatplotlibDeprecationWarning: The 'labels' parameter of boxplot() has been renamed 'tick_labels' since Matplotlib 3.9; support for the old name will be dropped in 3.11.
  box = plt.boxplot(
```



Задача 2. Анализ рынка акций (4 балла)

1. Есть набор файлов, каждый из которых содержит данные по одной из акций. Все файлы имеют одинаковую структуру, в том числе наименование столбцов. Загрузите данные в один dataframe.

В качестве значений индекса dataframe'а необходимо указать значения столбца "Date". Название столбцов должные соответствовать названию акций (имя файла без .csv), а их значения - значениям цены закрытия (столбец "Close" в файлах .csv)

Набор данных

⚠️ **Замечание.** Наборы данные были загружены с сайта https://finance.yahoo.com/

⚠️ **Замечание.** Нужна только цена закрытия

In [16]:
```
import os

folder_path = "./data/stock"

dfs = []

for file in os.listdir(folder_path):
    if file.endswith(".csv"):
        file_path = os.path.join(folder_path, file)

        stock_name = file.replace(".csv", "")

        df = pd.read_csv(file_path)
```

```
        df = df[["Date", "Close"]].copy()

        df["Date"] = pd.to_datetime(df["Date"])

        df.set_index("Date", inplace=True)

        df.rename(columns={"Close": stock_name}, inplace=True)

        dfs.append(df)

df_final = pd.concat(dfs, axis=1)

df_final.sort_index(inplace=True)

df_final.head()
```

Out[16]:

Date	AAPL	ABNB	ADBE	AMZN	CSCO	DBX	EBAY	GOOGL	GTLB	
2022-01-01	174.779999	153.970001	534.299988	149.573502	55.669998	24.750000	60.070000	135.303497	64.010002	36.7
2022-02-01	165.119995	151.490005	467.679993	153.563004	55.770000	22.690001	54.590000	135.057007	58.270000	34.3
2022-03-01	174.610001	171.759995	455.619995	162.997498	55.759998	23.250000	57.259998	139.067505	54.450001	36.2
2022-04-01	157.649994	153.210007	395.950012	124.281502	48.980000	21.750000	51.919998	114.109497	47.930000	36.6
2022-05-01	148.839996	120.870003	416.480011	120.209503	45.049999	20.840000	48.669998	113.762001	38.939999	38.8

5 rows × 25 columns

2. Рассчитайте корреляционную матрицу для всех акций

Корреляционная матрица по ценам

In [17]:

```
corr_matrix = df_final.corr()
corr_matrix
```

Out[17]:

	AAPL	ABNB	ADBE	AMZN	CSCO	DBX	EBAY	GOOGL	GTLB	HPQ	...
AAPL	1.000000	0.617430	0.833129	0.665715	0.589552	0.740429	0.115591	0.806847	0.282373	0.067074	...
ABNB	0.617430	1.000000	0.670509	0.830690	0.594365	0.332740	0.644140	0.780440	0.460602	0.390153	...
ADBE	0.833129	0.670509	1.000000	0.819614	0.554172	0.816359	0.180354	0.915440	0.496556	0.081518	...
AMZN	0.665715	0.830690	0.819614	1.000000	0.404820	0.478171	0.434078	0.912332	0.690644	0.235247	...
CSCO	0.589552	0.594365	0.554172	0.404820	1.000000	0.496982	0.494938	0.600025	0.068856	0.214262	...
DBX	0.740429	0.332740	0.816359	0.478171	0.496982	1.000000	-0.157363	0.669228	0.402517	-0.177013	...
EBAY	0.115591	0.644140	0.180354	0.434078	0.494938	-0.157363	1.000000	0.375794	0.251066	0.744560	...
GOOGL	0.806847	0.780440	0.915440	0.912332	0.600025	0.669228	0.375794	1.000000	0.535473	0.263251	...
GTLB	0.282373	0.460602	0.496556	0.690644	0.068856	0.402517	0.251066	0.535473	1.000000	0.094128	...
HPQ	0.067074	0.390153	0.081518	0.235247	0.214262	-0.177013	0.744560	0.263251	0.094128	1.000000	...
INTC	0.507251	0.738241	0.713875	0.816519	0.420854	0.390625	0.580047	0.826042	0.535441	0.591406	...
META	0.705358	0.723419	0.873388	0.830910	0.374998	0.552874	0.190361	0.808784	0.467641	-0.035611	...
MSFT	0.790691	0.679204	0.913842	0.838702	0.391476	0.648164	0.127010	0.845993	0.451366	-0.034581	...
MU	0.606787	0.842928	0.817961	0.906932	0.472688	0.440043	0.512637	0.867191	0.543109	0.308473	...
NFLX	0.701937	0.646901	0.821314	0.735466	0.497727	0.635239	0.138580	0.717756	0.452625	-0.203337	...
NVDA	0.633114	0.649664	0.802739	0.765294	0.320159	0.519374	0.087027	0.715287	0.404702	-0.160502	...
ORCL	0.769309	0.471504	0.785432	0.534556	0.463955	0.667833	-0.070414	0.618983	0.138574	-0.260316	...
PINS	0.640294	0.554616	0.804657	0.666996	0.384233	0.710191	-0.002757	0.640675	0.525458	-0.285950	...
SHOP	0.465147	0.696599	0.783919	0.824934	-0.144612	0.424923	0.338672	0.824313	0.855342	0.436406	...
SPOT	0.687415	0.753797	0.863827	0.875779	0.424007	0.525305	0.296858	0.821587	0.540113	0.005774	...
TCOM	0.439363	0.294269	0.533298	0.309545	0.257188	0.423136	-0.149330	0.322718	0.103614	-0.443806	...
TSLA	0.248385	0.353807	0.071508	0.302321	0.253808	0.037233	0.434899	0.326662	0.260908	0.568231	...
TWLO	0.042914	0.429915	0.067604	0.314869	0.383777	-0.113102	0.753732	0.315410	0.310273	0.728572	...
UBER	0.661323	0.680764	0.834611	0.796897	0.326346	0.595928	0.085736	0.737311	0.521399	-0.180970	...
XIACY	0.408747	0.564475	0.697612	0.654564	0.474311	0.382992	0.535223	0.680658	0.453669	0.378627	...

25 rows × 25 columns

Корреляционная матрица по доходностям

In [18]:

```
returns = df_final.pct_change()

corr_matrix_returns = returns.corr()
corr_matrix_returns
```

Out[18]:

	AAPL	ABNB	ADBE	AMZN	CSCO	DBX	EBAY	GOOGL	GTLB	HPQ	..
AAPL	1.000000	0.647446	0.731185	0.733110	0.511391	0.477592	0.601010	0.646388	0.158643	0.614992	.
ABNB	0.647446	1.000000	0.331655	0.602549	0.307032	0.182396	0.669479	0.404628	0.095758	0.453268	.
ADBE	0.731185	0.331655	1.000000	0.547367	0.605236	0.674230	0.502995	0.694949	0.130822	0.654086	.
AMZN	0.733110	0.602549	0.547367	1.000000	0.380183	0.308853	0.592936	0.750967	0.227888	0.348873	.
CSCO	0.511391	0.307032	0.605236	0.380183	1.000000	0.511588	0.468107	0.541677	0.061787	0.308522	.
DBX	0.477592	0.182396	0.674230	0.308853	0.511588	1.000000	0.183296	0.512300	0.379190	0.287810	.
EBAY	0.601010	0.669479	0.502995	0.592936	0.468107	0.183296	1.000000	0.509710	0.005764	0.622829	.
GOOGL	0.646388	0.404628	0.694949	0.750967	0.541677	0.512300	0.509710	1.000000	0.058544	0.433166	.
GTLB	0.158643	0.095758	0.130822	0.227888	0.061787	0.379190	0.005764	0.058544	1.000000	-0.171600	.
HPQ	0.614992	0.453268	0.654086	0.348873	0.308522	0.287810	0.622829	0.433166	-0.171600	1.000000	.
INTC	0.628958	0.329686	0.737019	0.470793	0.459761	0.389236	0.484077	0.694259	-0.094847	0.625513	.
META	0.283086	0.412975	0.317912	0.420852	0.077907	0.044582	0.447566	0.436819	0.017672	0.312165	.
MSFT	0.720323	0.344484	0.721935	0.681678	0.404698	0.406132	0.500548	0.717719	-0.014709	0.541489	.
MU	0.541623	0.518996	0.558090	0.623924	0.357455	0.168857	0.633857	0.695949	-0.176071	0.634437	.
NFLX	0.633480	0.387956	0.617213	0.705339	0.531426	0.464971	0.440985	0.589725	0.236143	0.324887	.
NVDA	0.628074	0.559473	0.610339	0.733313	0.594690	0.322448	0.639051	0.751810	0.051689	0.553061	.
ORCL	0.701751	0.331598	0.777190	0.435940	0.641939	0.510788	0.552421	0.488373	0.098257	0.617909	.
PINS	0.503623	0.400087	0.520749	0.431676	0.387163	0.456518	0.204845	0.357948	0.346293	0.202661	.
SHOP	0.825391	0.223743	0.674881	0.523197	-0.093108	0.469150	0.154927	0.562995	0.435462	0.607759	.
SPOT	0.590612	0.497531	0.579405	0.791144	0.313209	0.227043	0.583853	0.593886	0.177408	0.348503	.
TCOM	-0.210709	-0.042676	-0.031395	-0.074029	-0.007016	-0.043614	0.147777	0.051070	0.137351	0.002815	.
TSLA	0.688299	0.697942	0.294874	0.700233	0.262389	0.310343	0.463589	0.486235	0.289696	0.325166	.
TWLO	0.469787	0.369599	0.372659	0.457946	0.264862	0.345064	0.160562	0.458295	0.275446	0.284635	.
UBER	0.557328	0.690022	0.336555	0.599888	0.284072	0.220770	0.510545	0.475298	0.358598	0.294386	.
XIACY	0.194591	0.076747	0.466746	0.220979	0.262082	0.118740	0.425949	0.371537	0.096607	0.435135	.

25 rows × 25 columns

3. Отобразите корреляционную матрицу в виде диаграммы. Примерный вид графика приведен ниже

No description has been provided for this image

Диаграмма корреляционной матрицы по ценам

In [19]:

```
plt.figure(figsize=(10, 8))
plt.imshow(corr_matrix, interpolation='nearest')
plt.colorbar()

plt.xticks(range(len(corr_matrix.columns)), corr_matrix.columns, rotation=90)
plt.yticks(range(len(corr_matrix.columns)), corr_matrix.columns)

plt.title("Correlation Matrix")
plt.tight_layout()
plt.show()
```

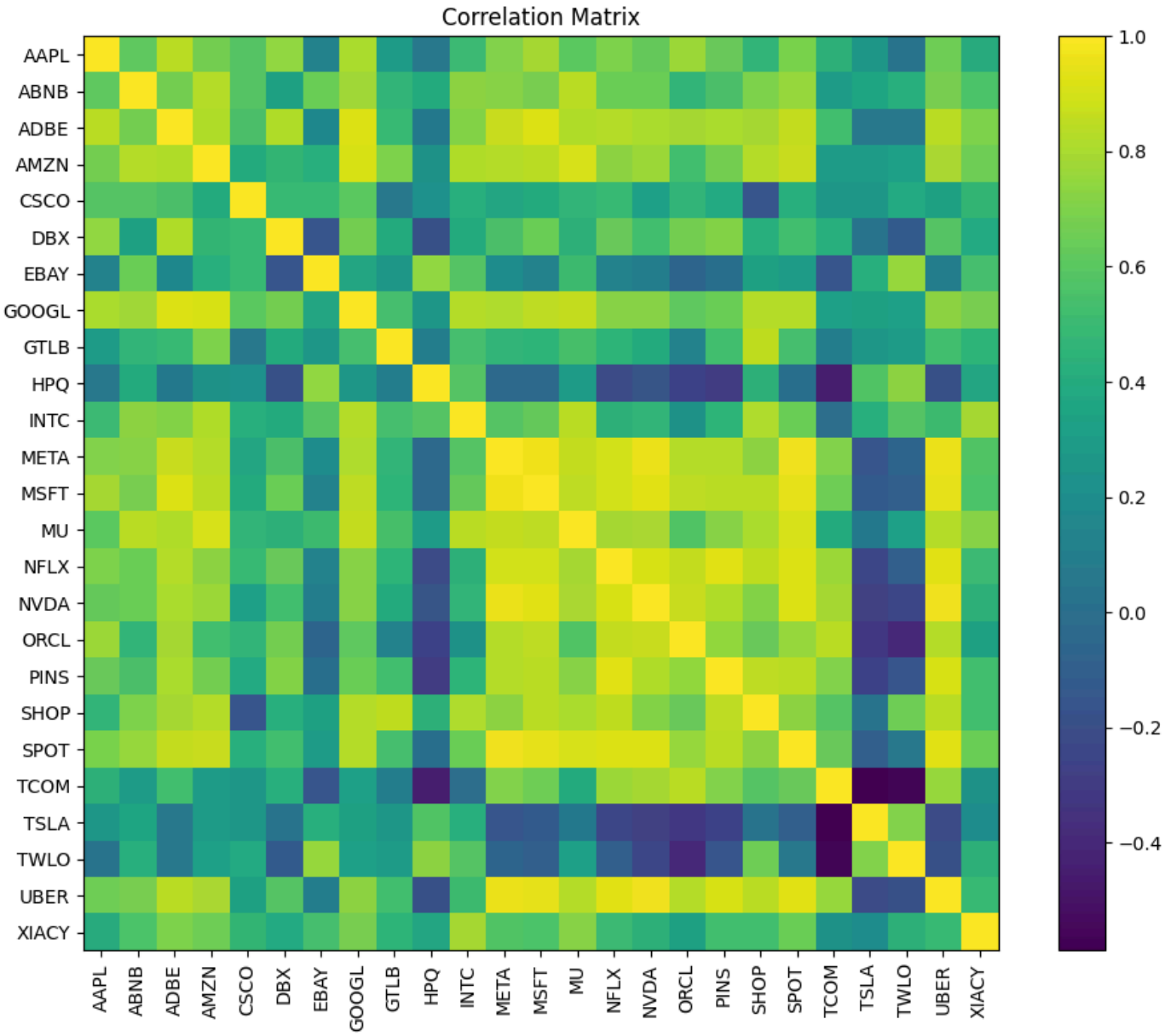
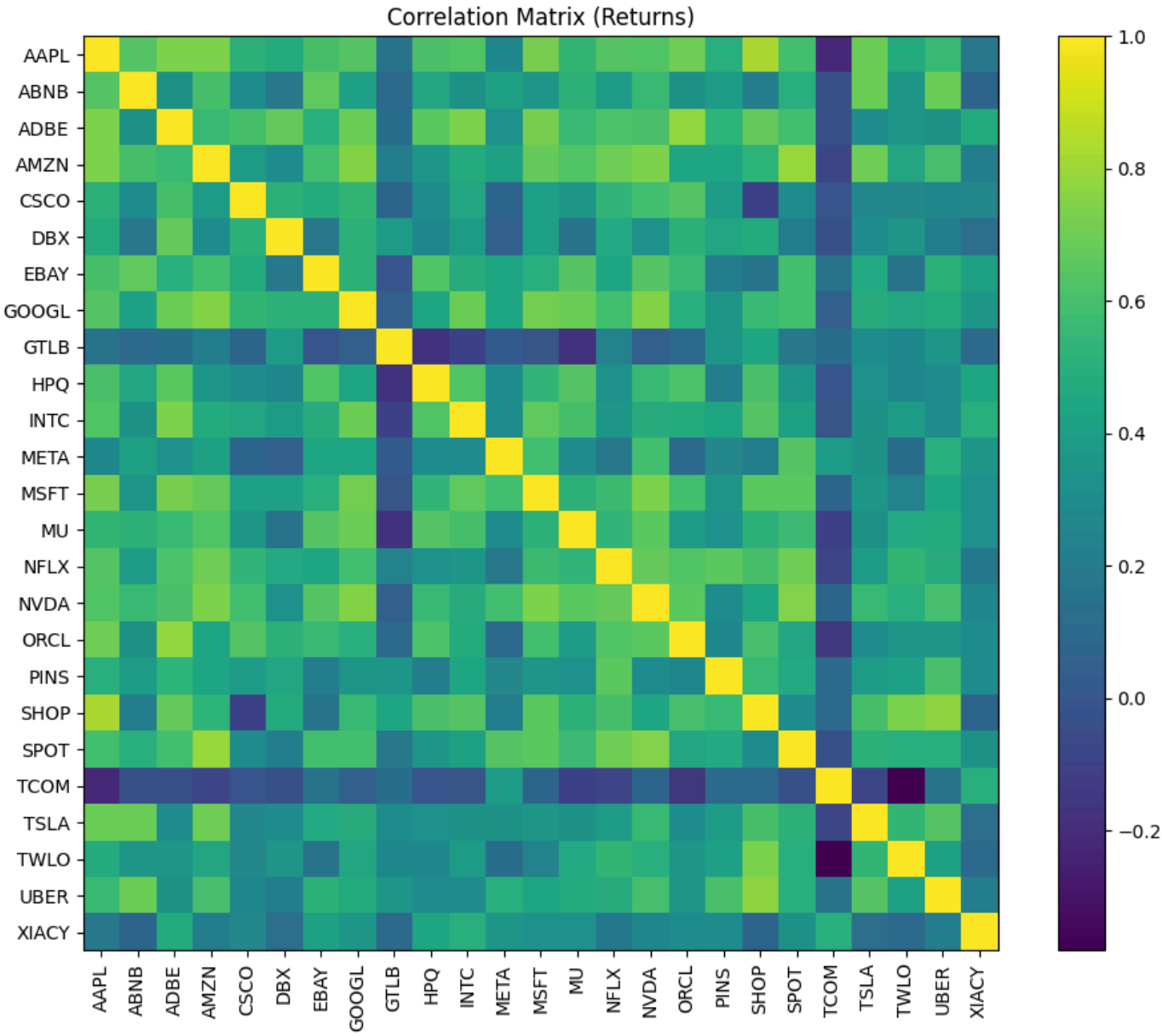


Диаграмма корреляционной матрицы по доходностям

In [20]:
```
plt.figure(figsize=(10, 8))
plt.imshow(corr_matrix_returns, interpolation='nearest')
plt.colorbar()

plt.xticks(range(len(corr_matrix_returns.columns)), corr_matrix_returns.columns, rotation=90)
plt.yticks(range(len(corr_matrix_returns.columns)), corr_matrix_returns.columns)

plt.title("Correlation Matrix (Returns)")
plt.tight_layout()
plt.show()
```



4. В соответствии с Вашим вариантом определите:

- акцию с максимальной положительной корреляцией (max)
- акцию с максимальной отрицательной корреляцией (min)
- акцию с минимальной корреляцией (которая больше всего соответствует отсутствию какой-либо корреляции (none)

Варианты

- **Вариант 1.** Microsoft (MSFT)
- **Вариант 2.** Netflix (NFLX)
- **Вариант 3.** Oracle (ORCL)
- **Вариант 4.** Uber (UBER)

***Вариант 3.** Oracle (ORCL)*

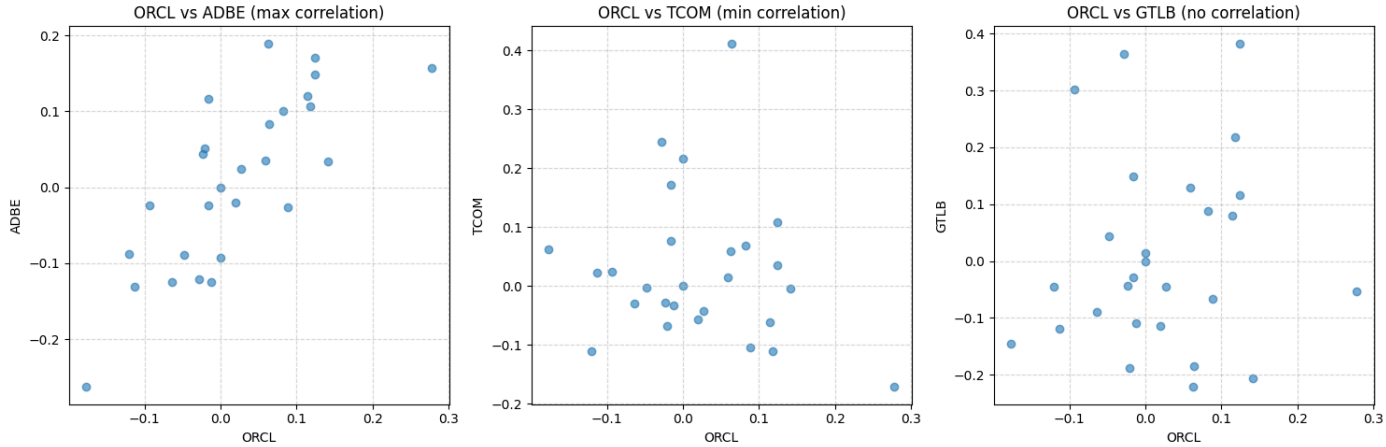
In [21]:
```
returns = df_final.pct_change()

corr_with_orcl = returns.corr()["ORCL"].drop("ORCL")
display(corr_with_orcl)

max_stock = corr_with_orcl.idxmax()
max_value = corr_with_orcl.max()

min_stock = corr_with_orcl.idxmin()
min_value = corr_with_orcl.min()
```

```
none_stock = corr_with_orcl.abs().idxmin()
none_value = corr_with_orcl[none_stock]

print("Max correlation:", max_stock, max_value)
print("Min correlation:", min_stock, min_value)
print("No correlation:", none_stock, none_value)
```

```
AAPL     0.701751
ABNB     0.331598
ADBE     0.777190
AMZN     0.435940
CSCO     0.641939
DBX      0.510788
EBAY     0.552421
GOOGL    0.488373
GTLB     0.098257
HPQ      0.617909
INTC     0.477276
META     0.114413
MSFT     0.594392
MU       0.376223
NFLX     0.627368
NVDA     0.655950
PINS     0.282608
SHOP     0.600479
SPOT     0.451664
TCOM    -0.157168
TSLA     0.301747
TWLO     0.354822
UBER     0.352406
XIACY    0.293053
Name: ORCL, dtype: float64
Max correlation: ADBE 0.7771898746240583
Min correlation: TCOM -0.15716808338529162
No correlation: GTLB 0.09825743680975435
```

5. Постройте диаграммы разброса (Ваша компания - Компания с min), (Ваша компания - Компания с max), (Ваша компания - Компания с none)

In [22]:
```
pairs = [
    ("ORCL", max_stock, f"ORCL vs {max_stock} (max correlation)"),
    ("ORCL", min_stock, f"ORCL vs {min_stock} (min correlation)"),
    ("ORCL", none_stock, f"ORCL vs {none_stock} (no correlation)")
]

plt.figure(figsize=(15, 5))

for i, (stock1, stock2, title) in enumerate(pairs, 1):
    plt.subplot(1, 3, i)
    plt.scatter(returns[stock1], returns[stock2], alpha=0.6)
    plt.xlabel(stock1)
    plt.ylabel(stock2)
    plt.title(title)
    plt.grid(True, linestyle='--', alpha=0.5)

plt.tight_layout()
plt.show()
```



6. Рассчитайте среднюю цену акций для каждого месяца (исходные данные взяты с интервалом в месяц)

In [23]:
```
df_final.index = pd.to_datetime(df_final.index)

monthly_mean = df_final.resample('ME').mean()

monthly_mean_all_stocks = monthly_mean.mean(axis=1)

monthly_mean["Average_All_Stocks"] = monthly_mean_all_stocks

display(monthly_mean_all_stocks.head())
```

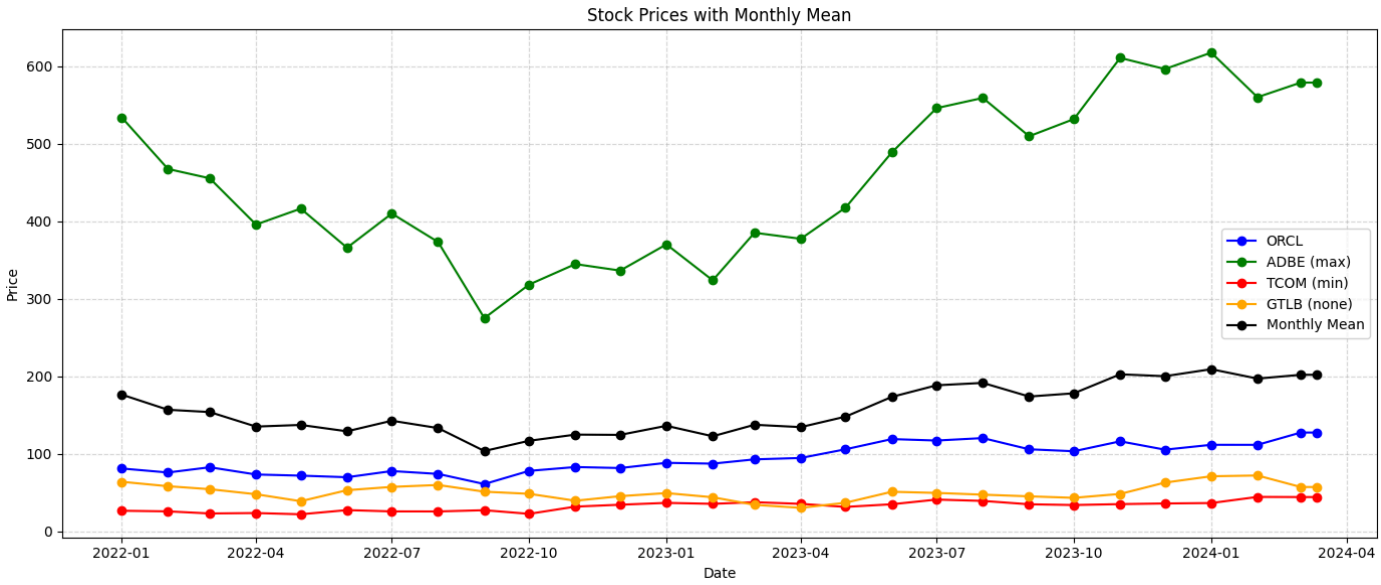
```
Date
2022-01-31    154.857167
2022-02-28    140.774723
2022-03-31    145.272287
2022-04-30    115.763514
2022-05-31    112.316034
Freq: ME, dtype: float64
```

7. Постройте графики для акций из пункта 4 и средней из пункта 6. Примерный вид графика приведен ниже

No description has been provided for this image

In [24]:
```
# Акции из пункта 4
my_stock = "ORCL"
max_stock = "ADBE"
min_stock = "TCOM"
none_stock = "GTLB"

df_plot = df_final[[my_stock, max_stock, min_stock, none_stock]].copy()
df_plot["Monthly Mean"] = df_plot.mean(axis=1)

colors = {
    my_stock: "blue",
    max_stock: "green",
    min_stock: "red",
    none_stock: "orange",
    "Monthly Mean": "black"
}

legend_labels = {
    my_stock: my_stock,
    max_stock: f"{max_stock} (max)",
    min_stock: f"{min_stock} (min)",
    none_stock: f"{none_stock} (none)",
    "Monthly Mean": "Monthly Mean"
}

if USE_PLOTLY:
    fig = go.Figure()
```

```
    for col in df_plot.columns:
        fig.add_trace(go.Scatter(
            x=df_plot.index,
            y=df_plot[col],
            mode='lines+markers',
            line=dict(color=colors[col], width=2),
            marker=dict(size=6),
            name=legend_labels[col],
            hovertemplate='<b>%{text}</b><br>Date: %{x}<br>Price: %{y:.2f}',
            text=[col]*len(df_plot)
        ))

    fig.update_layout(
        title="Stock Prices with Monthly Mean",
        xaxis_title="Date",
        yaxis_title="Price",
        template="plotly_white",
        hovermode="closest"
    )
    fig.show()

else:
    plt.figure(figsize=(14, 6))
    for col in df_plot.columns:
        plt.plot(
            df_plot.index,
            df_plot[col],
            marker='o',
            linestyle='-',
            label=legend_labels[col],
            color=colors[col]
        )

    plt.xlabel("Date")
    plt.ylabel("Price")
    plt.title("Stock Prices with Monthly Mean")
    plt.grid(True, linestyle='--', alpha=0.5)
    plt.legend()
    plt.tight_layout()
    plt.show()
```



Вывод

В ходе выполнения данной работы были успешно достигнуты все поставленные цели. Был получен практический опыт решения задач анализа данных, включая загрузку, преобразование информации, вычисление базовых статистических показателей, а также визуализацию результатов в виде графиков и диаграмм с использованием языка программирования Python. Кроме того, в процессе выполнения работы были освоены дополнительные практические навыки программирования на Python, такие как работа с

файлами, парсинг веб-страниц и экспорт данных в формат PDF, что расширило общий инструментарий для решения прикладных задач в области анализа данных.